

Integrating Independent Contributions in a Game Programming Assignment

Chao Peng*

cxpigm@rit.edu

School of Interactive Games and Media, Rochester
Institute of Technology
Rochester, New York, USA

David Schwartz

disvks@rit.edu

School of Interactive Games and Media, Rochester
Institute of Technology
Rochester, New York, USA

Lizhou Cao

lc1248@rit.edu

School of Interactive Games and Media, Rochester
Institute of Technology
Rochester, New York, USA

Huadong Zhang

hz2208@rit.edu

School of Interactive Games and Media, Rochester
Institute of Technology
Rochester, New York, USA

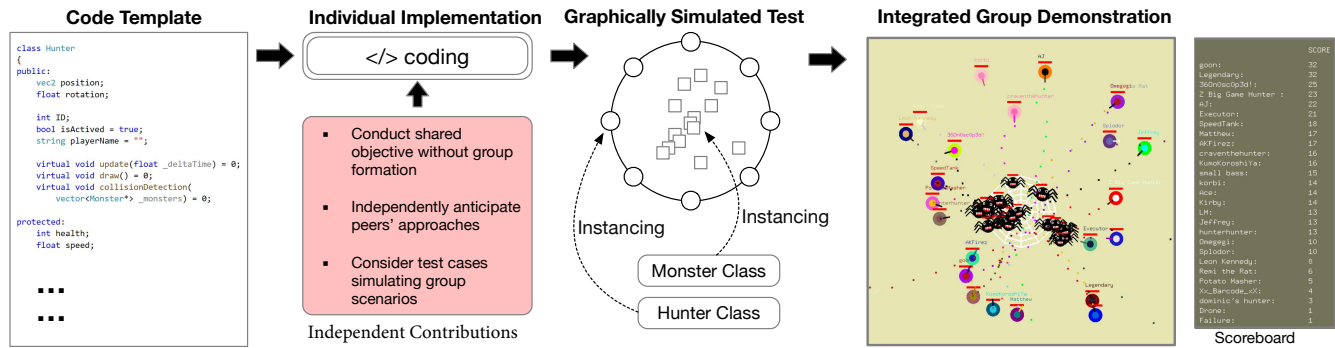


Figure 1: The game programming assignment integrates independent contributions. Students are tasked to develop individual implementations from a provided code template and refine them using a graphically simulated test supported by OpenGL. Their implementations are integrated into a group demonstration for assignment evaluation.

ABSTRACT

In game development curriculum, mastering programming skills is important. Traditional methods to teach programming in computer science curriculum often face challenges in relevance for game development students, leaning towards individual mastery of the knowledge. This has shown a sign of contrast with the collaborative nature of game design and development courses where teamwork is emphasized. In our approach, especially for a game programming assignment that is often integrated with computer graphics requirements, we introduce the strategy of embedding “imagined social elements” into individual programming assignments, simulating team dynamics without a formal team structure. Through this approach,

an assignment promotes the integration of independent contributions while maintaining the individual rigor essential for programming practices. The proposed assignment aims to be adopted in a game programming course. We have set up a git repository for this assignment (<https://github.com/LizhouCao/Integrating-Independent-Contributions-in-a-Game-Programming-Assignment.git>), with build instructions and required dependencies included.

CCS CONCEPTS

• **Social and professional topics** → **Computing education programs.**

KEYWORDS

Game programming assignment, integrating independent contributions

ACM Reference Format:

Chao Peng, Lizhou Cao, David Schwartz, and Huadong Zhang. 2024. Integrating Independent Contributions in a Game Programming Assignment. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH Educators Forum '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641235.3664431>

*corresponding author: cxpigm@rit.edu

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SIGGRAPH Educators Forum '24, July 27–August 01, 2024, Denver, CO, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0517-5/24/07
<https://doi.org/10.1145/3641235.3664431>

1 INTRODUCTION

The significance of learning programming in game development extends beyond traditional computer science education [Peng 2015]. Proficiency in programming skills, data structures, and algorithmic knowledge empowers game developers to implement and optimize game programs. The existing computer science-oriented programming assignments present challenges for students majoring in game development. The inherently abstract nature of data structure and algorithm (DSA) and the reliance on numerical examples as a primary mode of practice may pose impediments for students seeking to forge a meaningful connection between theoretical concepts and their practical application in the realm of game development. The discrepancy between the abstract representation of DSA in conventional computer science assignments and the tangible, interactive nature of game applications can hinder the students' ability to comprehend and apply programming principles effectively.

Furthermore, in game design and development courses, teamwork is integral [Echeverría et al. 2017]. They often employ a collaborative approach, which differs from many fundamental computer science courses, such as data structures and algorithms, that traditionally emphasize individual mastery through solo-focused methods. Recognizing the value of social interaction in game development, our approach integrates independent contributions to align DSA and programming courses more closely with other game design and development courses. This integration supports the combination of individual efforts into a group demonstration while preserving the individual rigor essential for mastering programming skills.

2 INTEGRATING INDEPENDENT CONTRIBUTIONS

This programming assignment entails a unique approach to teamwork where students contribute on a shared objective while not forming groups. In this scenario, individuals develop their own strategies to implement the tasks required in the assignment without knowledge of how their peers will actually approach the task. However, they should consider possible strategies that their peers could employ. The individual contribution unfolds as each student is tasked with creating their solo implementations. They anticipate potential scenarios, consider various test cases, and debug their implementations, enhancing the robustness of their strategies within the isolated environment of their individual work.

When all students submit their solo implementations, the instructor integrates these diverse efforts to run a comprehensive simulation. This simulation showcases the integrated independent contributions in action, as students witness the dynamic interactions and outcomes of their strategies when pitted against those of their classmates. This assignment lies in the shared experience of observing how each unique implementation interacts with others in the grand simulation.

3 PROGRAMMING ASSIGNMENT - HUNTERS VS. MONSTERS

Programming Language and Assignment Objective. We chose C++ with the OpenGL graphics API due to its proximity to low-level operations, making it ideal for exploring memory management

and algorithmic complexities [Chen 2010]. Before this assignment, students have learned the basics of data streaming between the CPU and GPU, the graphical rasterization pipeline, and the infinite loop structure of simulations, laying the groundwork for implementing the required algorithmic strategies.

Survival Game Story. Monsters have emerged, threatening the area. Hunters must unite to battle them. In this survival game, each student will develop their own “Hunter” class to survive and defeat as many monsters as possible. Students should consider their classmates' strategies while implementing their own plans to win the game.

Coding Template and Expected Efforts. A virtual class named “Hunter” was provided, allowing students to create individualized implementations based on its base features. This class includes essential functions like “render” for rendering the hunter. By using virtual functions, it enables dynamic polymorphism, requiring students to develop their own derived classes. For example, students complete the “update” function, which requires analyzing all instances of monsters and other hunters to implement logic for controlling their hunter's behavior in response to these entities.

Testing in Simulated Integrated Environment. To help students test their “Hunter” class, a function creates clones of their hunters around the monsters' nest. In the final group demonstration, these clones will be replaced with classmates' implementations.

4 RESULTS

We integrated all 30 student implementations into a single project by using polymorphism. This enables us to simulate diverse instances of students with varying behaviors within the same context and simulation environment.

During the class demonstration of the integrated project, we observed high engagement among students. This led to in-class discussions regarding their strategies, promoting students to reconsider their approaches. The demonstration sparked further discourse on potential improvements, as students revisited their strategies and implementations within this dynamic team simulation. To provide students with a live statistical result during the class demo, we implemented a side panel providing real-time updates on character information, including monster kills and hunter deaths. At the conclusion of the demo, this information was used for bonus grading.

In a rare case where the student's implementation deviated significantly (e.g., not using the provided class and function names), the course grader reviewed and rewrote the code in the same logic in order to align it with the team demo. This underscores the need for future considerations on advising students to adhere to the provided code template.

REFERENCES

- Hao Chen. 2010. Comparative Study of C, C++, C# and Java Programming Languages. (2010).
- Leovy Echeverría, Ruth Cobos, Liliana Machuca, and Ivan Claros. 2017. Using collaborative learning scenarios to teach programming to non-CS majors. *Computer applications in engineering education* 25, 5 (2017), 719–731.
- Chao Peng. 2015. Introductory game development course: A mix of programming and art. In *2015 International Conference on Computational Science and Computational Intelligence (CSCI)*. IEEE, 271–276.